Accelerating Bulk AES Encryption Performance in Telecommunications with GPU Computing

Marko Mihajlovic[1]

Abstract: This paper evaluates GPU computing for accelerating bulk AES encryption in telecommunications-oriented processing, where higher throughput must offset the cost of moving data through an accelerator. Validated AES-128 CTR and ECB implementations are compared with hardware-assisted CPU AES-NI across three heterogeneous systems, using paired workloads and five sessions per system. Measurements distinguish device-resident execution from a prepared-buffer pipeline including synchronous host–device transfers and synchronization. For 32 MiB CTR buffers, median paired pipeline speedups reach 1.354× on T600 and 4.761× on Tesla T4 relative to their respective CPU implementations, with gains in all five sessions. GTX achieves 0.935×, showing that the benefit depends on the configuration. These results demonstrate that GPU-assisted AES-CTR can outperform hardware-accelerated CPU encryption under the tested transfer-inclusive conditions, providing an experimental basis for bulk cryptographic processing in heterogeneous telecommunications architectures.

Keywords: Advanced Encryption Standard (AES), AES-NI, GPU acceleration, CUDA, heterogeneous computing, cryptographic benchmarking, data-transfer overhead, reproducible performance evaluation.

1 Introduction

Telecommunications infrastructure must protect increasing volumes of data while maintaining throughput and acceptable processing latency. Communication gateways, network services, and distributed processing systems therefore require efficient cryptographic operations. The Advanced Encryption Standard (AES), specified in NIST FIPS 197 [1], is a fundamental building block of symmetric encryption. Its practical performance depends on hardware acceleration, execution strategy, and data movement between processing resources, as well as the cryptographic algorithm.

Hardware-assisted CPU encryption provides the reference against which an alternative processing path must be evaluated. Graphics processing units (GPUs) offer parallel execution that can benefit workloads containing many independent AES operations. Counter (CTR) mode permits independent counter blocks to be encrypted in parallel to generate keystream [2], making it suitable for investigating GPU-assisted bulk encryption. Large buffers and collections of

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

independent data units are potential workloads for telecommunications-oriented systems that already contain general-purpose GPUs. Such systems could use existing heterogeneous computing resources for bulk encryption without requiring a dedicated cryptographic accelerator. Whether this is beneficial depends on the costs of moving the workload through the GPU as well as the throughput of the encryption operation itself.

Using these resources for encryption introduces host-to-device transfers, kernel execution, synchronization, and transfers back to host memory. These costs can reduce or eliminate the computational advantage of parallel execution. The relevant question is therefore whether GPU-assisted AES processing provides a reproducible bulk-throughput improvement over hardware-accelerated CPU execution, and which workload, platform, and data-movement conditions determine that improvement. The study distinguishes performance with data already resident in GPU memory from performance that includes synchronous host–device transfers and synchronization.

This distinction also affects the interpretation of potential processing strategies. Encrypting a contiguous buffer, aggregating independent requests, and operating on device-resident data involve different data-placement and execution costs. Batching may provide more parallel work per invocation, but its usefulness depends on workload size and application latency requirements. A favorable result for one execution boundary does not establish an advantage for the others; each must be assessed against the available host implementation.

The study evaluates validated AES-128 implementations in Electronic Codebook (ECB) and CTR modes across three heterogeneous computing configurations. CTR is the primary workload of interest, while ECB provides a controlled reference for independent block processing. Hardware-assisted CPU AES-NI serves as the baseline. Cryptographic correctness verification, controlled inputs, paired execution conditions, repeated measurement sessions, and explicit timing boundaries support the comparison. Section 3 specifies the implementations, workload sizes, and session-level analysis.

The contribution is experimental evidence identifying conditions under which GPU-assisted bulk AES processing improves throughput, including cases where an advantage remains after synchronous data movement. The measurements also identify configurations in which transfer costs or host implementation characteristics prevent consistent improvement. The common validation and measurement protocol provides a reproducible basis for assessing these outcomes across the tested systems. The study evaluates existing encryption approaches rather than introducing a new AES algorithm. Both favorable and unfavorable acceleration outcomes are used to delimit the operating conditions supported by the measurements. The results characterize the evaluated CPU and GPU paths; they do not establish a universal

hardware advantage or the maximum throughput attainable by either architecture.

For telecommunications, bulk data protection, workload aggregation, and device-resident processing are potential applications of these findings. Their application-level effectiveness remains untested. In particular, improved AES-CTR buffer processing does not demonstrate acceleration of complete network security protocols. Authenticated encryption additionally requires authentication processing, while practical protocols introduce packet handling, cryptographic state management, and data-delivery costs. These operations, along with batching and queueing latency, require separate evaluation before the measured bulk-processing improvements can be extended to telecommunications applications.

Section 2 reviews GPU AES implementations, CPU–GPU comparison methods, and the evidence needed to assess offloading. Section 3 describes the experimental design, validation, timing boundaries, and statistical analysis. Section 4 presents the measured acceleration outcomes, and Section 5 examines their practical constraints and potential telecommunications applications. Section 6 summarizes the findings.

2 Related Work and Research Gap

Early contributions by Manavski [3] and Harrison and Waldron [4] established the feasibility of AES execution on programmable GPU architectures. Subsequent work investigated how parallel processing, memory organization, and architecture-specific optimization affect encryption throughput. The available parallelism depends partly on the encryption mode: ECB permits independent block transformations, while CTR generates keystream from distinct counter values [2]. Both expose independent work for GPU threads when buffers are sufficiently large. ECB is unsuitable for protecting general structured data and serves here only as a controlled performance reference.

Iwai et al. [5] examined thread granularity, memory organization, and the interaction between computation and data transfers, including overlapping transfers with computation. Their CUDA implementation achieved approximately 35 Gbit/s for device-side AES processing and approximately 22.5 Gbit/s for transfer-aware execution. These results demonstrate that the execution boundary and transfer strategy affect the reported encryption rate. Wang and Chu [6] investigated lookup-table-based AES implementations and GPU execution strategies. Tezcan [7] examined shared-memory optimization and bank-conflict avoidance, reporting AES-128 encryption throughput of 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

More recent work has explored bit-sliced cipher representations. A 2024 study [8] reported high-performance AES-CTR and AES-ECB execution on desktop and embedded NVIDIA platforms. Together, these studies demonstrate the

computational capabilities of optimized GPU implementations. Their reported rates must nevertheless be interpreted within their respective measurement boundaries: device-side encryption throughput does not directly specify the rate at which an application can receive, encrypt, and deliver data. It also does not establish whether offloading improves performance relative to the host processor available in a particular system.

That comparison depends on the CPU implementation. Dedicated instructions such as AES-NI reduce the cost of AES transformations [9] and provide a hardware-assisted baseline for evaluating GPU offloading. Lee et al. [10] demonstrated that conclusions about heterogeneous acceleration depend strongly on the optimization level of the implementations being evaluated. Their findings support using an appropriately accelerated CPU reference when deciding whether GPU offloading offers an additional processing benefit. Accordingly, comparing GPU encryption with conventional unaccelerated software cannot establish an advantage over the host's hardware-assisted AES path. The reference must represent the execution path against which an offloading decision will actually be made, and the comparison remains specific to the implementations evaluated.

Data location further determines which costs enter that decision. Gregg and Hazelwood [11] showed that CPU–GPU performance evaluation requires explicit consideration of data movement. An advantage for device-resident input may shrink when both input and output cross the host–device boundary. CUDA guidance recommends minimizing unnecessary transfers, using appropriate synchronization, and considering overlap between communication and computation [12, 13]. The transfer-aware AES work of Iwai et al. [5] demonstrates that this concern is already established in GPU cryptographic processing.

For communication infrastructure, these findings connect encryption performance to the surrounding processing path. A host-managed interface may receive data, organize buffers, invoke encryption, and forward the output. The resulting costs differ between contiguous buffers, aggregated independent messages, and individual latency-sensitive packets. Device-resident measurements and transfer-inclusive measurements therefore answer different questions. The present study distinguishes resident execution from a synchronous prepared-buffer pipeline containing host-to-device and device-to-host transfers. This pipeline measures a broader operation than the kernel alone, but excludes complete network input/output and application processing; it also does not evaluate transfer–computation overlap.

Telecommunications security adds requirements beyond encryption throughput. AES-GCM combines counter-mode encryption with GHASH authentication [14]; an improvement in AES-CTR can therefore concern one component of an authenticated-encryption pipeline without establishing an equivalent improvement in IPsec or TLS processing. Practical implementations must also account for nonce

management, packet preparation, synchronization, and output delivery. These requirements distinguish a bulk encryption benchmark from a complete security service, even when the underlying cipher operations are parallelizable.

Earlier research also examined GPU-assisted encryption in communication protocols. Schönberger and Fuß [15] investigated GPU precomputation of AES counter-mode keystream for an IPsec-oriented GCM implementation, with subsequent encryption and authentication performed on the CPU. More recently, Lee et al. [16] developed a parallel GPU implementation of GCM, including GHASH processing, and reported throughput exceeding 400 Gbit/s on an RTX 4090. Thus, GPU-assisted telecommunications encryption and authenticated-encryption acceleration are established research directions. The present study instead characterizes reproducible operating conditions for bulk AES-128 processing, distinguishing device-resident execution from synchronous transfer-inclusive processing against hardware-assisted AES-NI. It neither introduces a new AES algorithm nor benchmarks complete AES-GCM or IPsec processing.

Workload aggregation introduces a further throughput–latency trade-off. Combining small messages may increase the work available to the GPU, while waiting for a batch adds queueing delay. Bulk data protection, traffic aggregation, heterogeneous communication gateways, and data-processing services are consequently possible application contexts, but their suitability depends on host acceleration, data placement, and operational requirements. Peak kernel rates alone cannot determine that suitability. The present measurements characterize contiguous bulk buffers and do not demonstrate packet batching or complete telecommunications protocol performance.

Assessing whether an observed advantage is repeatable also requires control of experimental variability. Mytkowicz et al. [17] showed that experimental conditions can introduce systematic bias and alter performance results. Kalibera and Jones [18] emphasized the importance of accounting for variability across multiple levels of experimental replication, while Hoefler and Belli [19] discussed scientifically defensible performance evaluation of parallel computing systems. These contributions provide a methodological basis for determining whether an acceleration is repeatable under the conditions tested. These studies motivate reporting repeated observations and their execution context rather than relying on the highest throughput measurement. For cryptographic workloads, functional correctness must additionally be verified before timings are accepted as evidence of encryption performance.

The present evaluation applies these principles through cryptographic validation, paired workloads, repeated study sessions, documented timing boundaries, and preservation of raw observations and experimental provenance. Session-level evidence distinguishes isolated favorable measurements from advantages observed

consistently across executions, while retaining outcomes that favor the CPU. Detailed procedures and aggregation equations are given in Section 3.

The literature thus establishes GPU AES feasibility, optimization techniques, and the need to account for transfers, CPU acceleration, and measurement variability. The question addressed here is which operating conditions produce a repeatable bulk-processing benefit when the host already supports hardware-assisted encryption. High kernel throughput alone cannot answer this question, and an unfavorable transfer-inclusive result on one system cannot settle it for other platforms or workloads.

To examine these conditions, the study compares AES-128 ECB and CTR workloads of different sizes across three heterogeneous configurations under a common validation and measurement protocol. The comparison uses hardware-assisted CPU AES-NI as its reference and retains session-level measurements for assessing consistency across repeated executions. Its contribution is the identification of experimentally supported operating conditions in which GPU-assisted bulk AES processing provides an advantage, together with evidence of constraints under which that advantage disappears or becomes inconsistent. This supports assessment of GPU-assisted bulk encryption and workload placement within the tested execution boundaries. Extending those findings to authenticated encryption and network processing requires separate application-level measurements.

3 Methodology

The evaluation compares AES-128 encryption using hardware-assisted CPU AES-NI and a validated CUDA implementation. CTR is the primary workload; ECB provides a controlled reference for mode-dependent implementation behavior. Both modes process contiguous buffers under the same validation and measurement protocol. Three complete hardware–software configurations are evaluated: two Windows systems and a Linux-based cloud allocation. Table 1 lists the recorded hardware and software. The exact model of the cloud host CPU was unavailable in the archived environment records.

The CPU reference uses PyCryptodome with hardware-assisted AES-NI explicitly enabled [9, 20]. Native backend initialization was checked for both modes; executed processor instructions were not traced directly. The GPU path uses the validated `cuda_baseline` kernel through CuPy's custom kernel interface [21]. Each CUDA thread processes one 16-byte AES block, with 128 threads per CUDA block. These paths are selected from a broader campaign that evaluated additional implementations. The analysis retains their original execution context, including interleaved jobs excluded from the selected results.

Cryptographic validation precedes acceptance of timing measurements.

Table 1
Hardware and software configurations used for AES acceleration experiments.

ID	Hardware and environment	Software stack
GTX	Intel Core i5-10300H; GeForce GTX 1650 Ti, 4 GB; Windows 10	Driver 610.62; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0
T600	Intel Core i7-11850H; T600 GPU, 4 GB; Windows 11	Driver 581.95; CUDA runtime 12.9; CuPy 14.2.0; PyCryptodome 3.24.1
T4	Intel Xeon at 2.00 GHz, two exposed logical processors; Tesla T4; Linux/Google Colab	Driver 580.82.07; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0

Known-answer tests and consistency checks against CPU reference operations use the AES specification and mode definitions [1, 2]. The archived records document successful CUDA execution and agreement with expected outputs. Executions failing the required checks are excluded from valid performance results. Experimental configuration, measurement-record completeness, and source-code identity are also verified.

The archived harness validates each measured output against the CPU reference outside the timed call, in addition to the initial validation gate. An output mismatch stops performance export. For each size, mode, and repetition, a single synthetic message is replayed across the paired backends. Plaintext and a 16-byte key are generated from the seeded random generator. The initial CTR block consists of eight generated bytes followed by eight zero bytes. The CPU CTR call uses an empty nonce and interprets the full initial counter as a big-endian integer.

Pair identifiers record size, mode, and repetition. A SHA-256 digest of the concatenated key, counter, and plaintext records the common input material. Raw records also retain the session identifier, seed, backend, mode, measurement scope, job order, global execution order, and validation flag. These fields allow the selected CPU and GPU observations to be matched without pooling observations from different sessions. Previous output buffers are released outside the next timed call.

The retained dataset is checked against the archived file hashes. For each platform, mode, size, and repetition, the selected observations contain one CPU transformation, one resident GPU measurement, and one pipeline measurement with matching input digests. Each configuration group contains all twenty recorded repetitions. Throughput is independently checked against the corresponding byte count and recorded time before session-level aggregation. Resident records specify one kernel iteration, and the GPU records identify pinned host memory. The

verification reconstructs the aggregates from raw records rather than relying solely on the saved summary tables.

The experiment uses five input sizes:

$$N \in \{1\ \text{KiB},\ 16\ \text{KiB},\ 1\ \text{MiB},\ 16\ \text{MiB},\ 32\ \text{MiB}\}. \tag{1}$$

Corresponding CPU and GPU executions use deterministic input generation and matching plaintext, encryption keys, and mode-specific parameters. Each platform has five study sessions with random seeds 2003–2007. For each measured configuration, five warm-up executions precede twenty recorded repetitions; warm-ups are excluded from the timing records. Each session contains thirty selected configuration groups, comprising two modes, five sizes, and three execution boundaries. With twenty observations per group, each session contributes 600 observations, giving

$$3 \times 5 \times 30 \times 20 = 9\,000 \tag{2}$$

raw timing observations across the three systems. Workload sizes execute in ascending order, while mode and backend/measurement-scope jobs are shuffled using the session seed. Benchmark jobs run sequentially. The selected observations were extracted from the existing campaign, without rerunning or reordering the benchmark schedule.

Three timing boundaries are retained:

1. **CPU AES-NI transformation.** The interval measures the host encryption call, including output allocation performed by that call. Cipher construction and key setup are excluded.
2. **GPU resident execution.** CUDA events measure a single kernel launch with input already in device memory. Host-to-device and device-to-host transfers are excluded.
3. **Prepared-buffer GPU pipeline.** The interval measures the synchronous host-driven sequence of host-to-device transfer, CUDA kernel execution, device-to-host transfer, and synchronization required to complete these operations.

CPU execution and the GPU pipeline are timed with Python's monotonic high-resolution `time.perf_counter_ns()` interface. Resident execution uses CUDA event timing with the required synchronization, following CUDA and CuPy measurement guidance [12, 22]. Thus, the CPU and pipeline intervals are measured from the host, while the resident interval measures device-side execution.

The prepared-buffer pipeline uses pinned host memory and preallocated device buffers, prepared before the timed interval. Its processing stages are represented as

$$T_{\text{pipeline}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}}, \tag{3}$$

where H2D and D2H denote host-to-device and device-to-host transfers, and the final term covers additional costs within the measured boundary. This expression describes the stages of the synchronous operation; it is not a decomposition calculated by adding separately aggregated component measurements. Recorded transfer-component intervals include synchronization effects and are not isolated measurements of physical PCIe transmission time.

GPU compilation, memory allocation and pinning, initial host-side input staging, key preparation, and application-level input/output are excluded from the pipeline interval. Transfers and computation are not overlapped using multiple asynchronous CUDA streams. Resident execution and prepared-buffer processing are measured separately, with the latter including the movement needed to process a host-resident buffer and return its output. No network input/output, packet aggregation, or authenticated-encryption operation is included in these measurements.

Elapsed processing time, throughput, and host-relative GPU acceleration are summarized at session level. For an input of N bytes and elapsed time $T_{s,r}$ in seconds at repetition r of session s, throughput in decimal MB/s is

$$R_{s,r} = \frac{N}{10^6 T_{s,r}}, \tag{4}$$

Input sizes use binary KiB and MiB units. Time and throughput medians are calculated independently within each session, then aggregated across the five session estimates:

$$\widetilde{T} = \text{median}_s \left(\text{median}_r \, T_{s,r} \right), \qquad \widetilde{R} = \text{median}_s \left(\text{median}_r \, R_{s,r} \right). \tag{5}$$

Throughput is summarized from the recorded per-repetition throughput values rather than obtained by inverting the aggregated latency statistic. For the corresponding CPU and GPU executions, acceleration is calculated from the paired session-level timing medians:

$$S_s = \frac{\widetilde{T}_{\text{AES-NI},s}}{\widetilde{T}_{\text{GPU},s}}, \qquad \widetilde{S} = \text{median}_{s=1,\ldots,5}(S_s). \tag{6}$$

Here, $\widetilde{T}_{\text{AES-NI},s}$ and $\widetilde{T}_{\text{GPU},s}$ denote within-session median times for the same platform, mode, and input size. GPU resident and pipeline speedups are calculated separately against the corresponding AES-NI time. Values of $S_s > 1$ indicate a faster GPU path, while values below one indicate a faster CPU reference. The reported $\widetilde{S}$ is the median of five paired ratios, not a ratio of independently aggregated CPU and GPU throughput values.

The analysis reports session ranges, interquartile ranges, sample coefficients of variation, and counts of sessions with $S_s > 1$. For five ordered session estimates, the interquartile range is the fourth minus the second value; the sample coefficient of variation is the sample standard deviation divided by the mean. An advantage in every session is recorded as $5/5$ directional consistency. These are descriptive statistics, not confidence intervals or formal significance tests. The twenty repetitions within a session are not treated as independent sessions, and measurements are not removed merely because they appear extreme.

The use of repeated sessions, paired workloads, and documented execution conditions follows recommendations on experimental bias and hierarchical variability in performance measurement [17–19]. Raw observations and their provenance are retained for verification of the session estimates and acceleration calculations.

4 Results

Figure 1 presents median paired GPU speedups across the five input sizes, separately for AES-128 ECB and CTR and for resident and prepared-buffer execution. At 32 MiB, resident execution exceeds the corresponding AES-NI reference on all three systems, in both modes and in every session. Pipeline results differ by platform and mode: CTR gains occur in all five sessions on T600 and T4, whereas the remaining platform–mode combinations have mixed or consistently slower pipeline results.

Table 2 provides an overview of the 32 MiB findings, with resident and pipeline medians, pipeline session ranges, and directional counts. Bold pipeline values identify improvements observed in all five sessions. The table retains both modes and all three systems, including cases with a median below parity and cases whose session range crosses parity.

For T600 CTR, the median pipeline speedup of 1.354× corresponds to approximately 26.14% lower latency, calculated from the median paired CPU-to-GPU timing ratio. Its session range remains above parity, from 1.129× to 1.391×. The paired speedup has an interquartile range of 0.030896 and a sample coefficient of variation of approximately 8.18%. These statistics describe the five session-level ratios used for this configuration.

T4 CTR has the largest median pipeline speedup at 32 MiB, 4.761×, with a

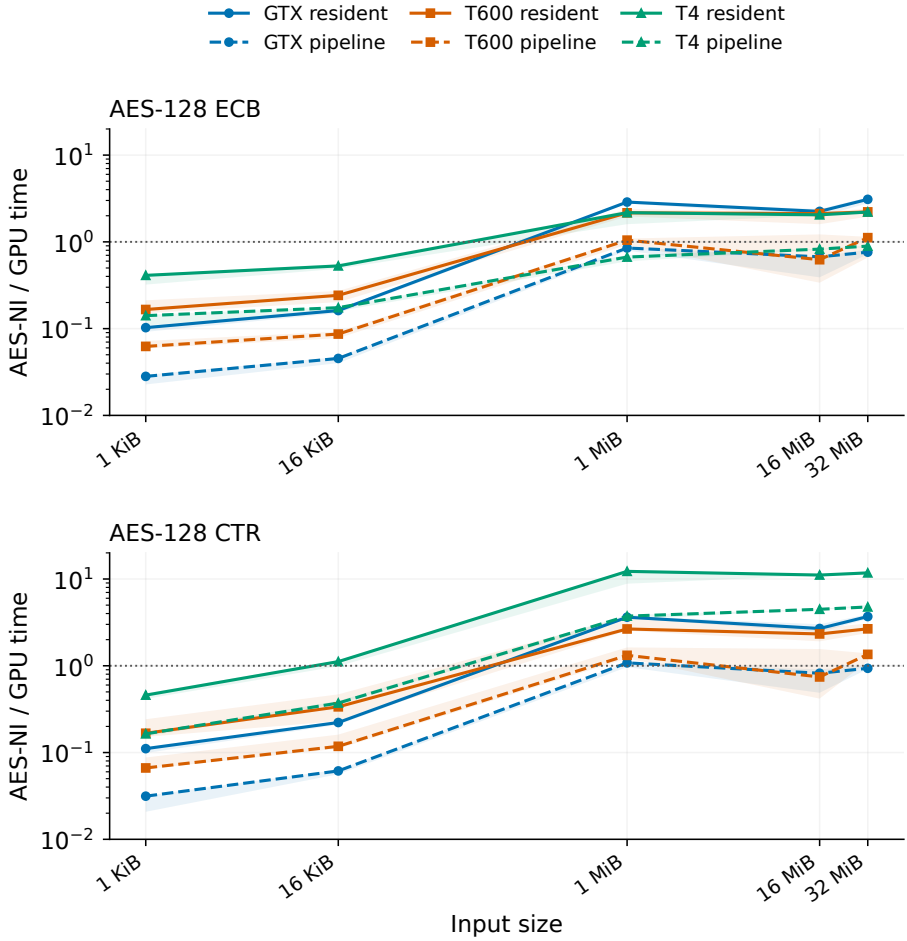


Fig. 1 — GPU-assisted AES-128 acceleration across input sizes and encryption modes. Curves represent median paired speedups relative to the corresponding host AES-NI implementation, with one panel per mode and distinct colors and markers for each GPU system. Solid lines denote resident execution; dashed lines denote the prepared-buffer pipeline. Resident execution measures device-side computation, whereas the prepared-buffer pipeline includes synchronous host–device transfers and synchronization. Shaded regions represent the observed ranges across five study sessions, not confidence intervals. The horizontal reference denotes parity.

Table 2
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

System	Mode	R	P	P range	$P > 1$
GTX	ECB	3.091	0.761	[0.722, 0.818]	0/5
GTX	CTR	3.693	0.935	[0.918, 1.008]	1/5
T600	ECB	2.215	1.118	[0.659, 1.144]	3/5
T600	CTR	2.666	**1.354**	[1.129, 1.391]	**5/5**
T4	ECB	2.207	0.893	[0.868, 0.901]	0/5
T4	CTR	11.758	**4.761**	[4.691, 5.021]	**5/5**

Bold pipeline medians and counts indicate acceleration in all five sessions. Resident execution is faster in all five sessions for every row.

five-session range of 4.691–5.021×. For both T600 and T4 CTR, every recorded session favors the pipeline even though the measured interval includes transfers and synchronization. The resident speedups are larger than the pipeline speedups on both systems, as they are for every platform–mode combination in Table 2.

GTX CTR has a pipeline median of 0.935× and favors the GPU in one session. Its range, 0.918–1.008×, crosses parity. GTX ECB remains below parity in all five sessions. T600 ECB has a median above one, 1.118×, but only three sessions favor the GPU, with a range of 0.659–1.144×. T4 ECB remains slower than AES-NI throughout the five sessions despite resident speedups above one. Thus, the positive T600 ECB median coexists with mixed session outcomes, while the T600 and T4 CTR medians accompany exclusively positive outcomes.

Table 3 reports absolute GPU throughput at the same input size. T4 has the highest resident and pipeline CTR rates among the tested systems. GTX resident CTR throughput exceeds T600 resident throughput, while the ordering is reversed for the pipeline. On each system, the pipeline rate is below the corresponding resident rate in both encryption modes.

The T4 GPU rates are nearly identical across modes: approximately 3713 MB/s for ECB and 3714 MB/s for CTR in the pipeline, and 9177 and 9167 MB/s, respectively, in resident execution. In contrast, its host-relative pipeline speedups are 0.893× for ECB and 4.761× for CTR. The host AES-NI ECB/CTR throughput ratio has a session median of 5.378× and a range of 5.250–5.795×. The corresponding host ratios on GTX and T600 are approximately 1.26×. The T4 host therefore records higher ECB than CTR throughput in every session, while its measured GPU throughput changes little between modes. Figure 2 compares the absolute rates of all three execution paths, using the same vertical scale for both modes.

The smaller workloads in Figure 1 show different acceleration outcomes. At

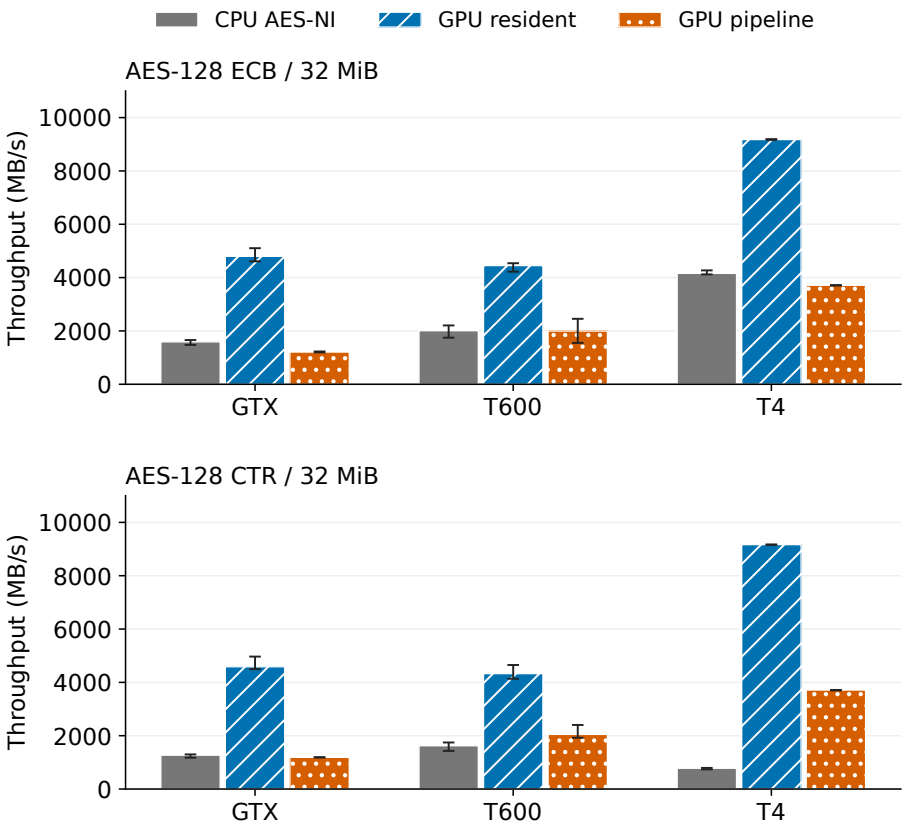


Fig. 2 — AES-128 throughput at 32 MiB for CPU AES-NI, resident GPU execution, and the prepared-buffer GPU pipeline. Bars show medians of five session-level throughput medians; error bars span the observed session minima and maxima, not confidence intervals. Throughput uses decimal MB/s. The GPU pipeline includes synchronous host–device transfers and synchronization; resident execution excludes these transfers. CPU bars represent the corresponding host implementation on each system.

Table 3
GPU throughput at 32 MiB in decimal MB/s. Values are medians of five session-level throughput medians.

System	Mode	Resident	Pipeline
GTX	ECB	4797.6	1207.8
GTX	CTR	4585.6	1188.9
T600	ECB	4451.1	1997.8
T600	CTR	4328.9	2049.4
T4	ECB	9176.9	3713.0
T4	CTR	9166.8	3713.9

1 KiB, both resident and pipeline paths are slower than AES-NI in every session, on all systems and in both modes. At 16 KiB, all pipeline configurations remain slower in every session. Resident T4 CTR has a median above parity at that size, with gains in four sessions; the other resident configurations remain below parity throughout. At 1, 16, and 32 MiB, resident execution is faster than AES-NI in all sessions for every platform and mode.

Table 4 summarizes CTR pipeline results across the size grid. T600 gains occur in all sessions at 1 and 32 MiB, but in only one session at 16 MiB. T4 gains occur in all sessions at each of the three MiB-scale inputs. GTX has four favorable sessions at 1 MiB, none at 16 MiB, and one at 32 MiB; no tested size has five favorable GTX pipeline sessions. ECB pipeline results likewise contain no five-session improvement on GTX or T4. T600 ECB favors the GPU in four, two, and three sessions at 1, 16, and 32 MiB, respectively.

Table 4
CTR pipeline speedup across input sizes. Each entry gives the median paired speedup and, in parentheses, the number of sessions faster than AES-NI out of five. Bold entries have gains in all five sessions.

Input	GTX	T600	T4
1 KiB	0.031 (0/5)	0.066 (0/5)	0.164 (0/5)
16 KiB	0.061 (0/5)	0.118 (0/5)	0.372 (0/5)
1 MiB	1.080 (4/5)	**1.320 (5/5)**	**3.734 (5/5)**
16 MiB	0.820 (0/5)	0.744 (1/5)	**4.478 (5/5)**
32 MiB	0.935 (1/5)	**1.354 (5/5)**	**4.761 (5/5)**

Absolute throughput is non-monotonic across the size grid. On GTX, throughput at 16 MiB is below both the 1 MiB and 32 MiB rates in all five sessions, for both modes and both GPU timing boundaries. On T4, this local reduction appears in resident throughput in four sessions for each mode, while neither pipeline mode

shows the same pattern. The session counts therefore differ between platforms and between the two timing boundaries on T4.

5 Discussion

5.1 Interpretation of Experimental Findings

The experiments show that GPU-assisted AES-CTR acceleration can persist beyond isolated kernel execution when synchronous host–device transfers and synchronization are included. The positive prepared-buffer results on T600 and T4 identify favorable bulk-processing conditions for the tested implementations. The T600 result also shows that the observed benefit is not confined to the datacenter-oriented configuration. The GTX pipeline and mode-dependent outcomes establish that resident computational performance alone does not determine whether offloading improves host-observed processing time.

CTR permits independent counter-block encryption, provided that counter and nonce requirements are satisfied [2]. This independence supplies parallel work for the CUDA implementation. At 32 MiB, the resident measurements show an advantage over the evaluated AES-NI path on every system in both modes. The pipeline measurements then establish whether an advantage remains within the broader execution boundary. Their lower throughput reflects the additional operations in that boundary, but the measurements do not isolate how much of the difference is attributable to physical transfers, synchronization, or runtime overhead.

The host implementation is equally relevant to interpreting acceleration. On T4, similar GPU throughput in ECB and CTR accompanies different host-relative speedups because the corresponding AES-NI rates differ substantially. The CTR speedup therefore characterizes the complete CPU–GPU comparison; it does not indicate that the GPU executes CTR several times faster than ECB. The underlying cause of the host disparity was not isolated, so attributing it to a processor feature, library behavior, or cloud mechanism would exceed the evidence. The T600 and T4 acceleration factors also use different host processors as their denominators. Their relative magnitudes cannot rank GPU performance independently of the host; absolute throughput and host-relative acceleration must be considered together when comparing the tested configurations.

Workload size and session consistency further qualify these findings. Non-monotonic throughput prevents identification of a universal CPU-to-GPU crossover threshold. A positive aggregate speedup can also coexist with sessions favoring the CPU, as observed for T600 ECB. The results support selecting execution paths using workload-specific measurements and paired session outcomes, rather than

peak kernel throughput or a single favorable observation. They do not establish that 32 MiB is optimal or that larger buffers would necessarily improve performance.

5.2 Practical Implications and Generalizability

For telecommunications, the findings provide a basis for evaluating GPU-assisted bulk encryption in communication gateways and server-side data-protection services. Systems already handling substantial contiguous buffers could assess the GPU as an additional execution path alongside AES-NI. This is a potential use of the measured capability, not a demonstrated improvement in a deployed communication service. The relevant comparison must include the operations needed to acquire, organize, protect, and deliver data in the intended application. Where general-purpose GPUs are already present, this could allow bulk encryption to use existing heterogeneous resources without a separate cryptographic accelerator. The measurements support assessing that option against the available hardware-assisted CPU path rather than assuming that an additional accelerator will improve the service.

Aggregating independent requests could increase the parallel work available per accelerator invocation, but introduces requirements absent from the buffer experiments. Queueing while a batch forms can increase latency, especially under low traffic. Packet metadata, independent cryptographic contexts, counter allocation, and output ordering must also be maintained. Consequently, a contiguous 32 MiB buffer is not interchangeable with a collection of independently managed packets, and the measurements establish neither an optimal batching policy nor a minimum packet batch size. The relevant trade-off is therefore between the processing benefit of a larger GPU workload and the time spent waiting to assemble and manage it. Requests with strict latency requirements may need a different execution policy from sustained bulk traffic.

AES-CTR is specified for IPsec Encapsulating Security Payload (ESP) confidentiality in RFC 3686 [23], while AES-GCM is specified for ESP in RFC 4106 [24]. These standards connect the evaluated encryption operation to IPsec-oriented architectures, but the experiments cover neither complete ESP processing nor authenticated encryption. Such processing adds packet framing, security-association state, nonce and sequence-number management, authentication, and delivery. AES-GCM additionally requires GHASH and authentication-tag processing. The measured CTR acceleration cannot determine the throughput of that complete operation.

An IPsec extension must preserve counter uniqueness, authentication boundaries, and the association between packet data and protocol metadata when organizing parallel work. Its performance assessment would need to include both throughput and latency across the packet-processing path. Faster counter-mode

encryption could benefit that path only to the extent that the remaining operations and their interactions permit an overall improvement.

Device-resident processing offers another possible arrangement when input data are already available on the GPU. The resident measurements characterize encryption under this placement, while application acquisition, authentication, and delivery remain to be evaluated. Similar questions could arise in other data-intensive industries, such as storage or analytics services processing large buffers. These are plausible extensions of the workload structure, not validated application results. A workload-aware architecture retaining CPU processing for smaller or latency-sensitive requests and considering GPU execution for bulk work remains an untested design implication.

5.3 Study Limitations

Each configuration represents one host or cloud allocation with five study sessions. The observations establish repeatability within those environments, not performance across a population of comparable machines. Differences among the hardware and software configurations were not independently controlled, preventing causal attribution to a single component. The selected AES-NI library path and validated CUDA kernel also do not establish the fastest attainable CPU or GPU implementations, including multithreaded CPU services.

The prepared-buffer boundary excludes application-level work and preparation costs specified in Section 3. Its throughput is therefore not complete telecommunications-system throughput. The fixed ascending size order confounds buffer-size effects with possible time-dependent execution behavior; the local 16 MiB reduction cannot be assigned to memory effects, scheduling, clocks, or thermal conditions from these measurements. Five-session ranges and directional consistency are descriptive evidence rather than confidence intervals or formal significance tests. These limits constrain both causal explanations and predictions of performance beyond the tested execution paths.

5.4 Future Research

Three extensions follow directly from the measured acceleration and its constraints.

First, implement and evaluate an IPsec-oriented authenticated-encryption pipeline, including AES-GCM, GHASH, tag processing, and protocol state. A representative gateway workload should include packet preparation and delivery and compare complete throughput and latency with a hardware-assisted CPU implementation. This would test whether the observed encryption-component advantage survives the additional work required by the protocol.

Second, evaluate packet aggregation and workload-aware CPU–GPU selection under varying traffic volume and latency requirements. The study should retain independent cryptographic contexts and output ordering while measuring queueing delay as well as processing time. More densely sampled workload sizes and additional independent systems would help assess whether a selection policy remains effective beyond the current size grid and host allocations.

Third, compare the synchronous pipeline with overlapping transfers and computation, persistent device-resident processing, and supported direct-data-transfer mechanisms. Measurements should separate the relevant processing stages while also reporting the complete application interval. This would determine whether changing data placement and transfer scheduling alters the conditions favoring GPU execution; the present synchronous results alone do not predict those gains.

6 Conclusion

This study experimentally compared validated GPU-assisted AES-128 ECB and CTR encryption with hardware-assisted CPU AES-NI across three heterogeneous configurations. It distinguished device-resident execution from a prepared-buffer pipeline containing synchronous host–device transfers and synchronization, using paired measurements across repeated sessions.

For 32 MiB AES-CTR buffers, the pipeline achieved median paired accelerations of 1.354× on T600 and 4.761× on T4 relative to their respective host implementations. Both improvements occurred in all five sessions, including on the tested 4 GB T600 GPU. Resident execution outperformed AES-NI at this size on every system in both modes, but the GTX pipeline did not show a consistent advantage, and T4 ECB remained slower than its CPU reference. Thus, a computational advantage survived data movement for selected CTR workloads, while the outcome depended on the system, mode, and execution boundary.

These findings establish measured operating conditions for GPU-assisted bulk encryption alongside existing CPU acceleration. They could inform workload placement in telecommunications gateways and other services handling large data buffers, including storage and analytics applications. Their relevance rests on matching the application's workload and data placement to the evaluated conditions. The experiments do not establish improved IPsec ESP, AES-GCM, packet-batching, or complete application performance. Future work should explore an IPsec-oriented authenticated-encryption prototype evaluated across the full packet-processing path. Measuring its throughput and latency with authentication, protocol state, packet aggregation, and delivery included would determine whether the demonstrated bulk AES-CTR improvement translates into an application-level

benefit.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023. URL https://doi.org/10.6028/NIST.FIPS.197-upd1.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001. URL https://doi.org/10.6028/NIST.SP.800-38A.

[3] Svetlin A. Manavski. CUDA compatible GPU as an efficient hardware accelerator for AES cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, pages 65–68, 2007. doi: 10.1109/ICSPC.2007.4728256.

[4] Owen Harrison and John Waldron. AES encryption and decryption on the GPU. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu.

[5] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of AES encryption on CUDA GPU. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[6] Canhui Wang and Xiaowen Chu. GPU accelerated AES algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[7] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[8] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of AES-CTR and AES-ECB bit-sliced implementation on GPUs. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[9] Intel Corporation. Intel advanced encryption standard instructions (AES-NI), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100X GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[11] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate CPU vs. GPU performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[12] NVIDIA Corporation. CUDA C++ best practices guide, n.d. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[13] Mark Harris. How to overlap data transfers in CUDA C/C++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[14] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode

(GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007. URL https://doi.org/10.6028/NIST.SP.800-38D.

[15] Georg Schönberger and Jürgen Fuß. GPU-assisted AES encryption using GCM. In *Communications and Multimedia Security*, volume 7025 of *Lecture Notes in Computer Science*, pages 178–185. Springer, 2011. doi: 10.1007/978-3-642-24712-5_16.

[16] JaeSeok Lee, DongCheon Kim, and Seog Chung Seo. Parallel implementation of GCM on GPUs. *ICT Express*, 11(2):310–316, 2025. doi: 10.1016/j.icte.2025.01.006.

[17] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[18] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.

[19] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[20] PyCryptodome. PyCryptodome documentation, n.d. URL https://www.pycryptodome.org/. Accessed 2026.

[21] CuPy. cupy.RawKernel, n.d.. URL https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html. Accessed 2026.

[22] CuPy. Performance best practices, n.d.. URL https://docs.cupy.dev/en/stable/user_guide/performance.html. Accessed 2026.

[23] Russ Housley. Using advanced encryption standard (AES) counter mode with IPsec encapsulating security payload (ESP). RFC 3686, Internet Engineering Task Force, January 2004. URL https://doi.org/10.17487/RFC3686.

[24] John Viega and David A. McGrew. The use of galois/counter mode (GCM) in IPsec encapsulating security payload (ESP). RFC 4106, Internet Engineering Task Force, June 2005. URL https://doi.org/10.17487/RFC4106.